Shipment Portal (Angular): Setup and Explanation

- Shipment Portal — Angular Guide & Interview Story
  - 1. Your 30-second pitch
  - 2. The story — one user's journey
  - 3. Where each Angular concept is used
  - 4. Explaining the key parts
  - 5. Likely interview questions about this project
  - 6. Live demo script (5 minutes)
  - 7. What I'd improve next
- Screens
- Setup Guide (README)
  - 1. What you need
  - 2. Setup (first time)
  - 3. Run everything (3 terminals)
  - 4. Configuration
  - 5. Other commands
  - 6. Pages
  - 7. Project structure
  - 8. Troubleshooting

Shipment Portal — Angular Guide & Interview Story

This guide explains the Angular app in simple words: the story of how it works, where each Angular concept is used, and what to say in an interview.

---

1. Your 30-second pitch

> "On top of my Laravel shipment API, I built an Angular 21 front end. It uses standalone components, signals for state, functional guards and interceptors, lazy-loaded routes, and reactive forms.
>
> The parts I like most: the booking form shows a **live price** using RxJS `debounceTime` and `switchMap`, so only the latest request counts. Laravel's 422 validation errors appear under the correct input. And the detail page **refreshes by itself** while the Laravel queue worker books the parcel in the background, so you can actually *see* the queue job

> working.
>
> All permissions come from Laravel's policy through a `can` block in the JSON, so the front end never repeats the business rules."

---

2. The story — one user's journey

Tell it like this, page by page. Each step shows one or two Angular ideas.

Step 1 — The app starts

`main.ts` starts the app with `appConfig`. Before the first page shows, **`provideAppInitializer`** runs `AuthService.restoreSession()`. If a token is saved in localStorage, the app asks Laravel `GET /auth/me` who it belongs to. That's why you stay logged in after refreshing the page.

Step 2 — A guest opens the site

The router wants `/dashboard`, but **`authGuard`** sees nobody is logged in. It sends the user to `/login?returnUrl=/dashboard`. After login, the app sends the user back to that address.

Step 3 — Logging in

The **reactive form** checks the email format on the client. On submit, `AuthService.login()` calls Laravel, saves the token, and sets the `currentUser` **signal**. Every component that reads `auth.user()` updates on its own — the header shows the name and role badge.

If the password is wrong, Laravel answers **422**. `applyServerErrors()` puts the message right under the email field.

From now on, **`authInterceptor`** adds `Authorization: Bearer <token>` to every API request automatically.

Step 4 — The dashboard

`toSignal(shipments.dashboard())` loads the counts. `computed()` builds the five cards and the "still on the way" number from that one signal. Clicking a card opens `/shipments?status=pending`.

Step 5 — Searching the list

The list page is one RxJS pipeline:

```
type "sar" ... "sara"
   -> debounceTime(300)          wait until typing stops
   -> distinctUntilChanged       ignore if nothing really changed
   -> switchMap(page stream)     new filters restart at page 1
   -> switchMap(http.get)        cancel the old request, keep only the newest
   -> toSignal                   the template just reads result()
```

Why switchMap? If the user types fast, an old slow response could arrive *after* a newer one and show the wrong results. `switchMap` cancels the old request, so that can never happen.

Step 6 — Booking a shipment

The form has the same rules as Laravel's `StoreShipmentRequest`, plus a **custom validator** for Pakistani mobile numbers. As soon as the weight or the cities change, the **live price** box asks Laravel for a quote (debounced, and switchMap again). The price always comes from the server, so it can't be faked in the browser.

On submit, Laravel returns **201**, and the app navigates to the detail page.

Step 7 — Watching the queue job

The new shipment is `pending`, because Laravel's queue worker is booking it with the courier in the background. The detail page has an `effect()`: while the status is `pending`, it reloads every 3 seconds. A few seconds later the badge changes to **"Booked with courier"** and the courier reference appears.

> This is a great live demo: it shows the Laravel job, the observer, the event and the listener all working, seen from the front end.

Step 8 — Cancelling

The Cancel button only shows if `can.cancel` is true, a value that comes from Laravel's policy. The click goes through `exhaustMap`, so a double-click sends only **one** request.

Step 9 — The admin

The admin sees an extra `StatusChangerComponent` (a child component). The parent gives it the current status with an **input**. The child sends the chosen status back with an **output**. The parent calls the API. If the move isn't allowed, Laravel's 422 message is shown in a toast.

The history timeline is wrapped in `@defer (on viewport)`, so its code only loads when it scrolls into view.

---

3. Where each Angular concept is used

Concept	Where	One-line explanation
Standalone components	every component	No NgModules. Each component lists its own `imports`.
`inject()`	services, guards, interceptors, components	DI without constructor parameters; works in functions too.
Signals: `signal`, `computed`, `asReadonly`	`AuthService`, `ToastService`, pages	Simple reactive state. The template updates when the value changes.
`effect()`	`ShipmentDetailComponent`	Auto-refresh while the status is pending, with cleanup.
`toSignal` / `toObservable`	dashboard, list, create, detail	Bridge between RxJS and signals. Unsubscribes automatically.
`input()`, `input.required()`, `output()`	`StatusBadge`, `StatusChanger`, `HistoryTimeline`, detail `id`	Parent → child data in, child → parent events out.
`withComponentInputBinding`	`app.config.ts`	The `:id` route parameter arrives as a component input.
Functional guards	`auth.guard.ts`	`authGuard`, `guestGuard` with `returnUrl`.
Functional interceptors	`auth.interceptor.ts`, `error.interceptor.ts`	Add the token; handle 0/401/403/429/500 in one place.
`provideAppInitializer`	`app.config.ts`	Restore the session before the first page.
Lazy loading	`app.routes.ts`, `shipments.routes.ts`	`loadComponent` and `loadChildren`. Each page is its own chunk.
Reactive forms	login, register, create, list filters	`NonNullableFormBuilder`, typed controls.
Custom validators	`shared/forms/validators.ts`	`pakistanMobile` (control) and `matchFields` (group).
Server-error mapping	`shared/forms/server-errors.ts`	Laravel 422 `errors` → `control.setErrors({ server })`.
RxJS operators	list, create, detail	`debounceTime`, `distinctUntilChanged`, `switchMap`, `exhaustMap`, `combineLatest`, `startWith`, `catchError`, `finalize`, `takeUntilDestroyed`.

OnPush change detection	almost every component	Faster rendering. Works because state is in signals.
New control flow	all templates	`@if`, `@for` (with `track`), `@else if`, `@empty`.
`@defer`	detail page	Lazy-load the history timeline when visible.
Custom pipe	`PkrPipe`	`{{ 1250 | pkr }}` → `Rs 1,250` (pure pipe).
Built-in pipe	`DatePipe`	Formats dates from Laravel's ISO strings.
Route `title`	routes	Browser tab title per page.
Environments	`src/environments/`	Different API URL for dev and production.
Unit tests (Vitest)	`*.spec.ts`	Pipe, validators, and the interceptor (with `HttpTestingController`).

---

4. Explaining the key parts

4.1 Why signals and RxJS together?

- **Signals** hold *state*: who is logged in, the current shipment, "is it saving?". They are simple to read in a template: `auth.user()`.
- **RxJS** handles *events over time*: typing, HTTP requests, clicks that may overlap.
- `toSignal()` turns an RxJS stream into a signal for the template, so there's no manual `subscribe` / `unsubscribe`.

Say: "I use the simplest tool for each job. Signals for state, RxJS for async flows where timing matters."

4.2 The four "map" operators (a very common question)

Operator	What it does with overlapping requests	Where in this app
`switchMap`	Cancels the old one, keeps the newest	Search list, live price, loading a shipment
`exhaustMap`	Ignores new ones while one is running	Cancel button (stops double-clicks)
`concatMap`	Waits and runs them in order	Not needed here; would be used for ordered saves
`mergeMap`	Runs them all at once	Not needed here

4.3 Interceptors — why two?

- `authInterceptor` only **adds** things: the token and the JSON header. It only does this for **our** API URL, so the token is never leaked to other websites. There's a test for that.
- `errorInterceptor` only **reacts** to errors. A 401 logs the user out, 403/429/500 show a toast, and 0 means the server is down. **422 is passed back** to the form, because only the form knows which field each message belongs to.

Keeping them separate follows the Single Responsibility principle, and makes each one easy to test.

4.4 Security: what the front end does and doesn't do

- Guards and hidden buttons are **user experience only**. Anyone can call the API directly with Postman.
- Real security is in Laravel: `auth:sanctum`, the `role:admin` middleware, and `ShipmentPolicy`.
- The front end reads `can.cancel` and `can.update_status` from the API, so it never duplicates the rules. If a rule changes in Laravel, the UI follows automatically.
- Angular escapes everything in `{{ }}` automatically, which protects against XSS.

Follow-up — "Why localStorage for the token?" — It's simple and works across tabs. The risk is XSS: a script injected into the page could read it. Angular's escaping reduces that risk. For higher security, Sanctum's **cookie-based SPA login** (HttpOnly cookie + CSRF) is the alternative, because JavaScript can't read the cookie at all.

4.5 Change detection

Almost every component uses `ChangeDetectionStrategy.OnPush`. Angular then only re-checks a component when an input changes, an event happens inside it, or a signal it reads changes. Because the state lives in signals and arrays are updated immutably (`[...list, toast]`), this works safely.

One exception: `FieldErrorComponent` uses default change detection on purpose. A form control's `touched` state isn't a signal or an input, so an OnPush component wouldn't notice it. (You can mention this — it shows you understand *why* OnPush works, not just that it's faster.)

4.6 Lazy loading

Only the root, the shell and the guards are in the first bundle. Login, the dashboard and each shipment page are separate files that download the first time they're needed. The production build's first load is about **96 kB gzipped**.

---

5. Likely interview questions about this project

Q: What happens if the Laravel API is down? The error interceptor gets status 0 and shows "Cannot reach the server". Each page also has its own `catchError`, so it shows a friendly message instead of a blank screen.

Q: How do you avoid memory leaks? I use `toSignal()` (it unsubscribes on destroy) and `takeUntilDestroyed()` for manual subscriptions. HTTP requests complete on their own anyway. The `effect()` uses `onCleanup` to clear its timer.

Q: How do you keep front-end and back-end validation in sync? The client validates for speed with the same rules. The server is the source of truth, and its 422 messages are mapped onto the same form fields, so the user sees one consistent place for errors.

Q: Why not NgRx? The state here is small: the current user, plus page data. Signals in a service are enough and much less code. I'd add NgRx (or the NgRx Signal Store) if many features started sharing complex state.

Q: How would you show live status without polling? Laravel can broadcast the `ShipmentStatusChanged` event over WebSockets (Laravel Reverb), and Angular would listen with Laravel Echo. Polling is fine for a demo because it only runs while a shipment is pending.

Q: How did you test it? Unit tests (Vitest) for the pipe, the validators, and the auth interceptor. The interceptor tests check that the token is added for our API and **not** sent to other websites. I also tested the full flow in a browser against the API.

---

6. Live demo script (5 minutes)

1. Open `/dashboard` while logged out → you're redirected to login (**guard**).
2. Enter a wrong password → the error appears under the email field (**422 mapping**).
3. Click **Customer** → **Log in** → a toast and the dashboard (**signals, toSignal, computed**).
4. Click the **Pending** card → the list is filtered (**query params**).
5. Type a name in search → results update after typing stops (**debounceTime + switchMap**).
6. **New shipment** → enter a short phone number → client error. Change the weight → the price updates (**live quote**).
7. Book it → watch "pending" turn into "Booked with courier" without touching the page (**effect + the Laravel queue worker**).
8. Log out → log in as **Admin** → open a shipment → move it to "In transit" (**child component input/output, role from the policy**).
9. Refresh the browser → still logged in (**app initializer + /auth/me**).

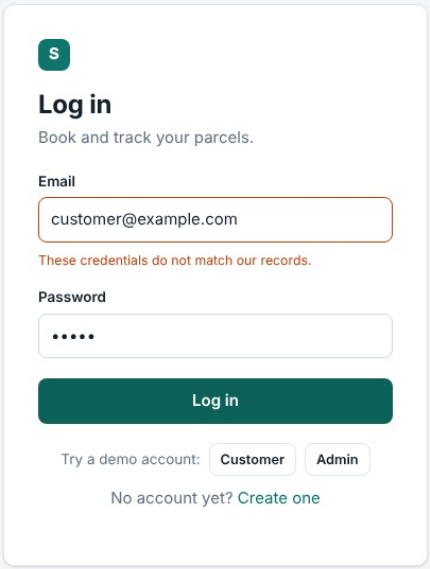
---

7. What I’d improve next

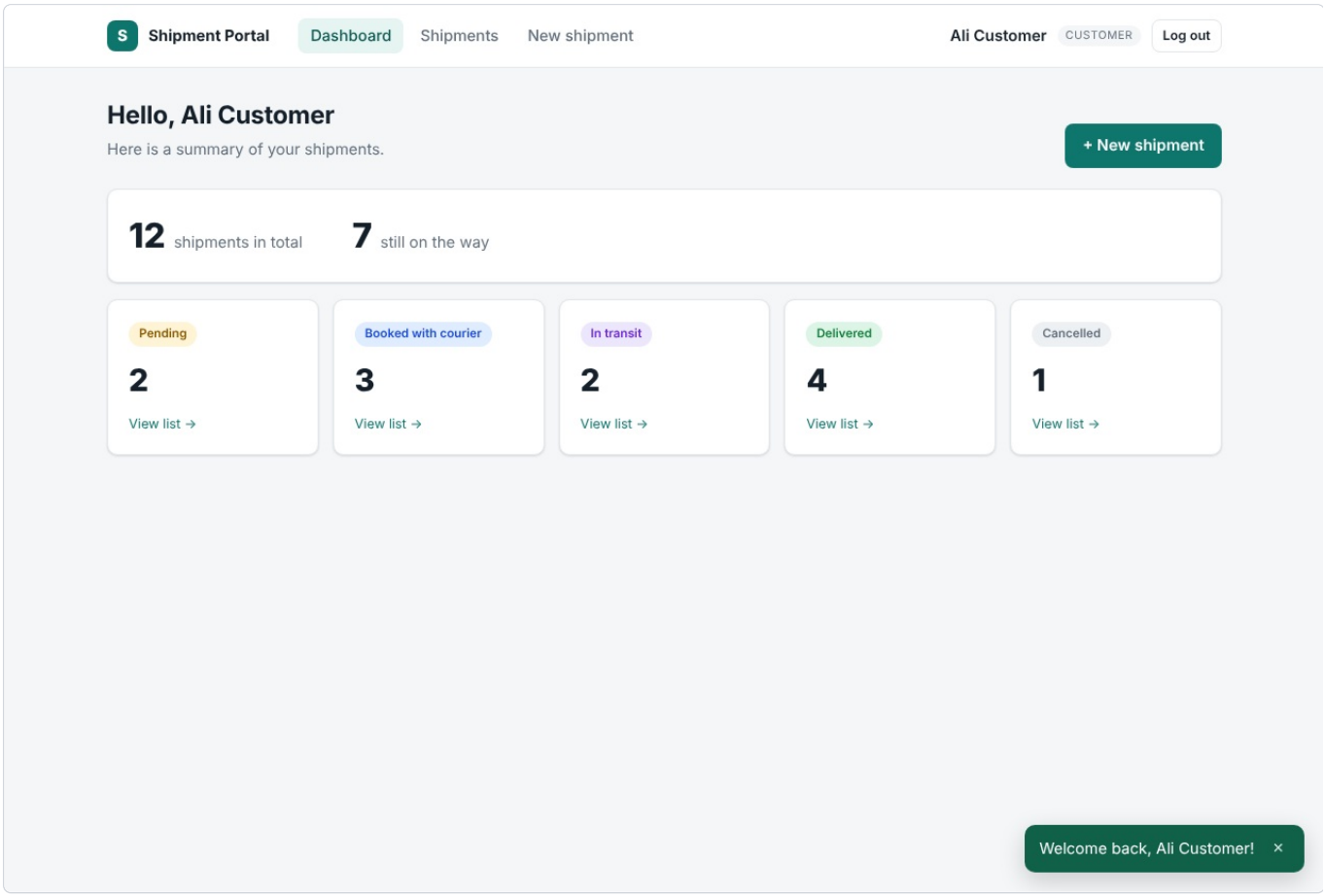
1. **Cookie-based Sanctum SPA auth** instead of localStorage tokens.
2. **Real-time updates** with Laravel Reverb + Echo instead of polling.
3. A **shipment form draft** saved in the URL or storage, so a refresh doesn’t lose input.
4. **End-to-end tests** with Playwright for the demo script above.
5. **i18n** (English/Urdu) with Angular’s localisation tools.

Screens

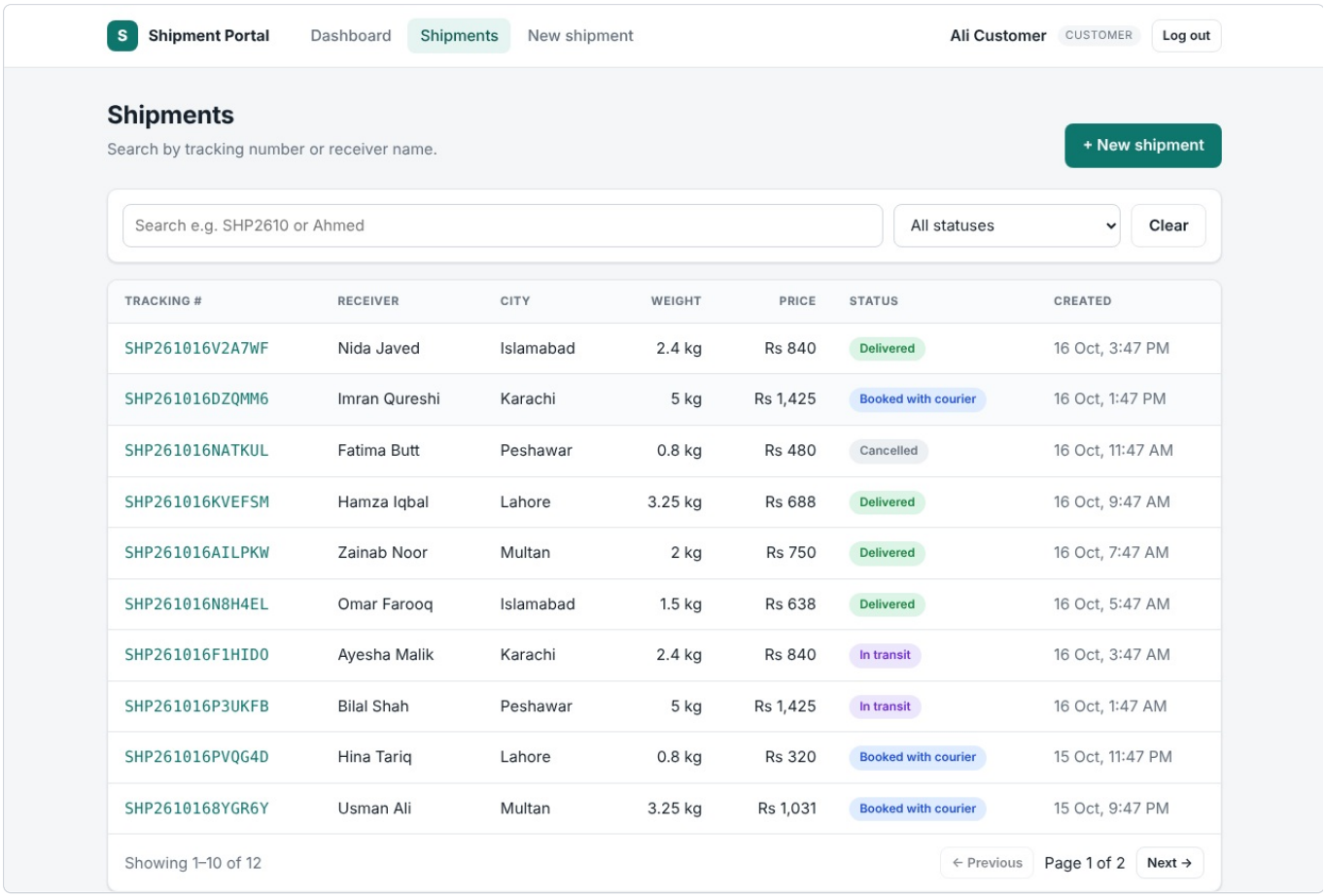
Login with a server validation error



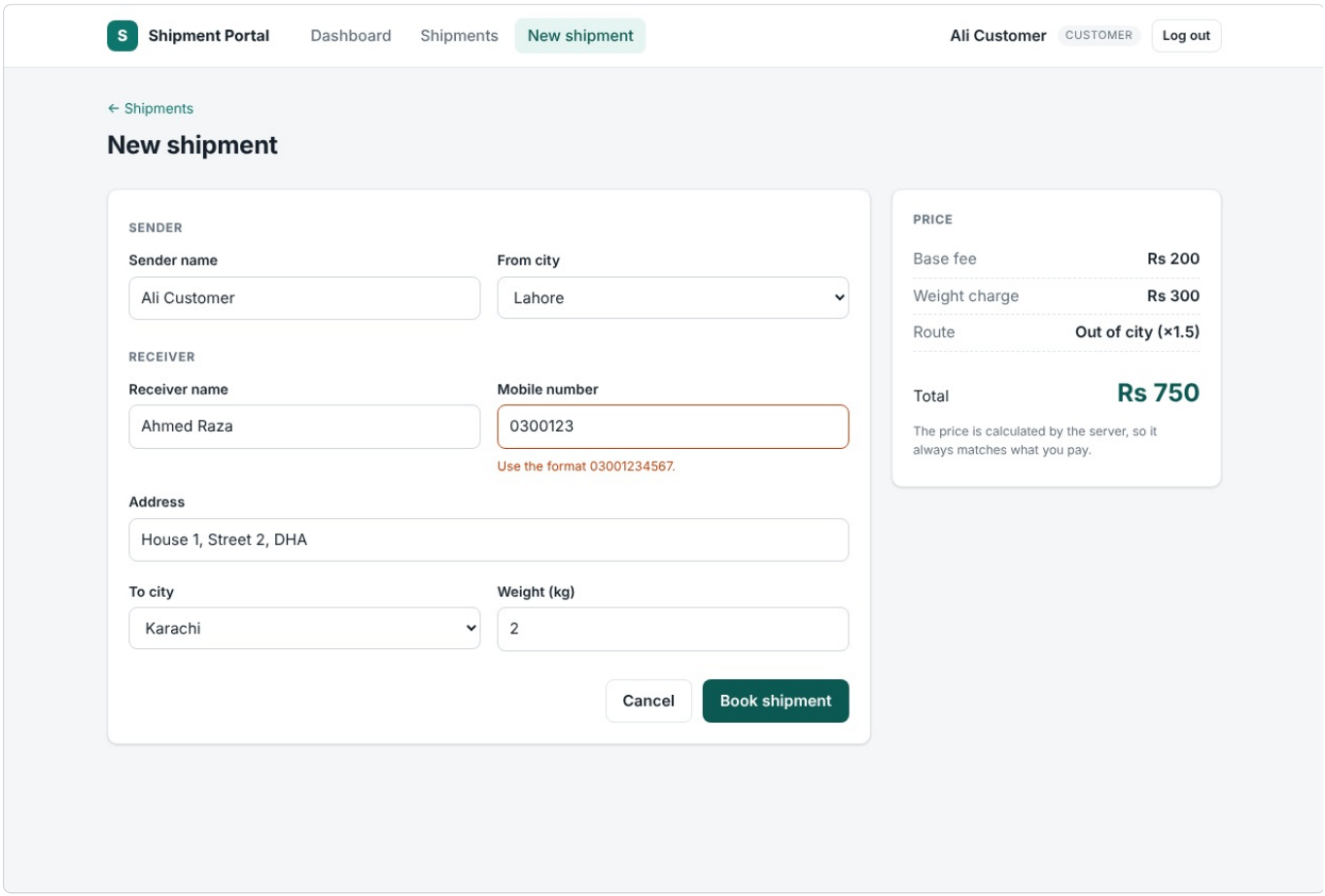
Dashboard


S
Shipment Portal
Dashboard
Shipments
New shipment
Ali Customer
CUSTOMER
Log out
Hello, Ali Customer
Here is a summary of your shipments.
+ New shipment
12 shipments in total
7 still on the way
Pending
2
View list →
Booked with courier
3
View list →
In transit
2
View list →
Delivered
4
View list →
Cancelled
1
View list →
Welcome back, Ali Customer!
×

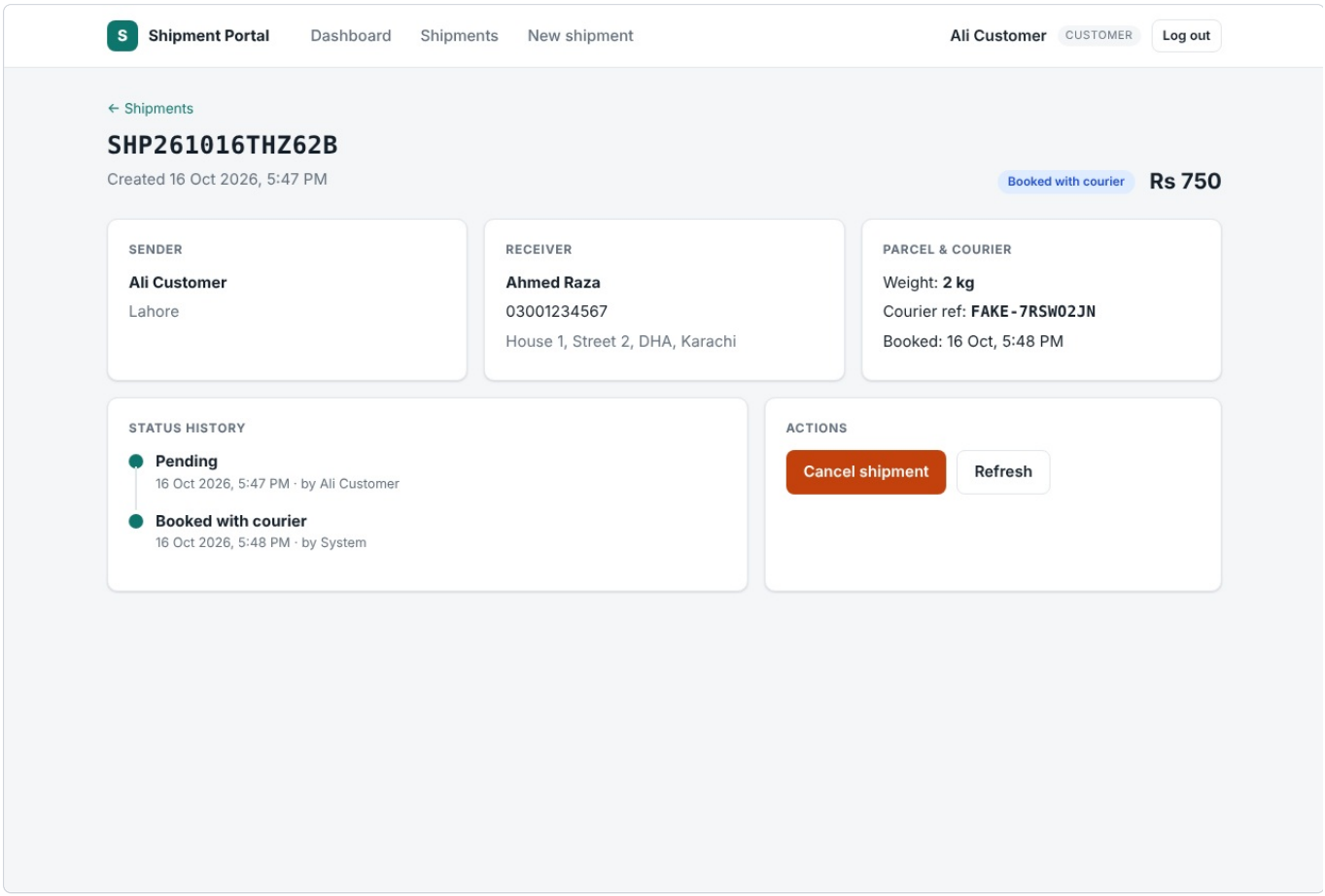
Shipments list



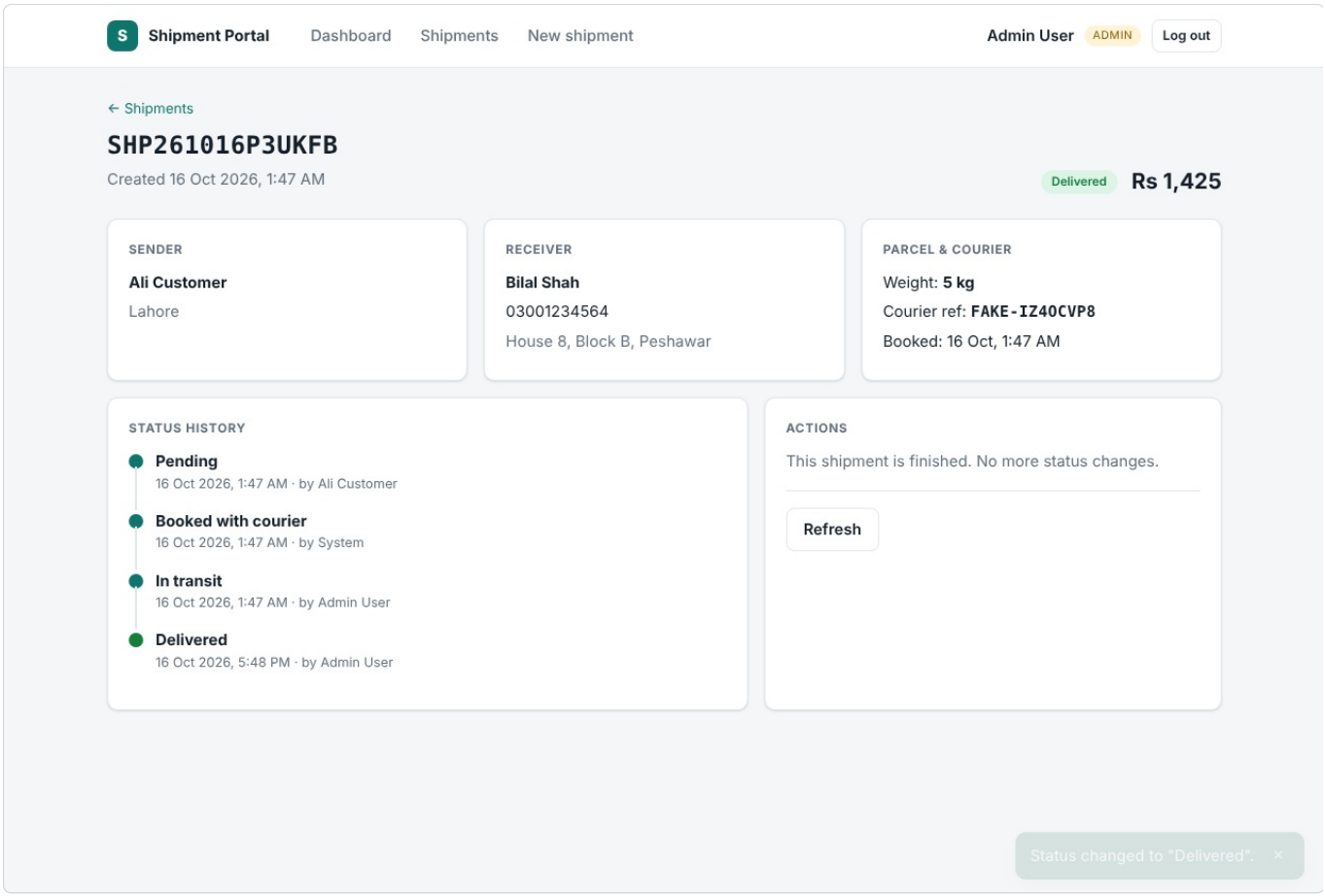
New shipment with live price and validation



Detail page after the queue job booked it



Admin changing the status



Setup Guide (README)

The Angular app for the **Shipment API** (Laravel 13 + Sanctum). Customers log in, see a dashboard, search their shipments, book a new one with a live price, and watch it get booked by the Laravel queue worker. Admins can move a shipment through its statuses.

> Want the explanation and interview story? Read **docs/ANGULAR_GUIDE.md**.

---

1. What you need

Tool	Version
Node.js	**20.19+** or **22.12+** (check with `node -v`)
npm	10+
The Laravel API	the `shipment-api` project, running on `http://localhost:8000`

You don't need to install the Angular CLI globally. `npx ng` and `npm start` use the version in the project.

2. Setup (first time)

```
cd shipment-portal
npm install
```

The project includes an `.npmrc` with `legacy-peer-deps=true`. This avoids a known peer-dependency bug in some npm 10 versions, so a plain `npm install` works.

3. Run everything (3 terminals)

```
# Terminal 1 — Laravel API (in the shipment-api folder)
php artisan serve                                          # http://localhost:8000

# Terminal 2 — Laravel queue worker (in the shipment-api folder)
php artisan queue:work --queue=courier,notifications,default

# Terminal 3 — Angular (in the shipment-portal folder)
npm start                                                  # http://localhost:4200
```

Open **http://localhost:4200** and use a demo account. The login page has one-click buttons for both.

Role	Email	Password
Customer	customer@example.com	password
Admin	admin@example.com	password

> The Laravel database must be seeded first (`php artisan migrate --seed`), otherwise the demo users don't exist.

4. Configuration

The API address is set in the environment files:

File	Used by	`apiUrl`
`src/environments/environment.development.ts`	`npm start` / `ng serve`	`http://localhost:8000/api`
`src/environments/environment.ts`	`npm run build` (production)	`/api` (same domain as the app)

`angular.json` swaps them automatically (`fileReplacements`).

On the Laravel side, `FRONTEND_URL=http://localhost:4200` in `.env` lets the browser call the API (CORS). If you run Angular on another port, change it there too.

5. Other commands

```
npm test          # unit tests (Vitest): 3 files, 8 tests
npm run build     # production build -> dist/shipment-portal/browser
```

To deploy, copy `dist/shipment-portal/browser` to a web server, and send every unknown URL to `index.html` so Angular's router can handle it.

6. Pages

URL	Page	Who
`/login`, `/register`	Log in / create account	guests only (`guestGuard`)
`/dashboard`	Totals and a card per status. Click a card to open the filtered list.	logged in (`authGuard`)
`/shipments`	Search, status filter and pagination	logged in
`/shipments/new`	Booking form with a live price from the server	logged in
`/shipments/:id`	Details, status history, Cancel button, admin status changer. Refreshes by itself while the status is “pending”.	owner or admin

7. Project structure

```
src/
  environments/                    API URL for dev and production
  app/
    app.config.ts                  router, HttpClient + interceptors, app
initializer
    app.routes.ts                  routes, guards, lazy loading
    app.ts                         root: <router-outlet> + toasts
    core/                          singletons used everywhere
      auth/        auth.service.ts      who is logged in (signals),
login/register/logout
                   auth.guard.ts        authGuard, guestGuard
                   token-storage.service.ts
      interceptors/ auth.interceptor.ts  adds "Authorization: Bearer <token>"
                   error.interceptor.ts 401 / 403 / 429 / 500 handled in one
place
      models/                      TypeScript types that match the Laravel JSON
      services/    shipment.service.ts  every API call
                   toast.service.ts     notifications
    layout/        shell.component      header + nav around logged-in pages
    features/      auth/  dashboard/  shipments/   (one folder per feature, lazy
loaded)
    shared/        components/  forms/  pipes/     (reusable pieces)
```

8. Troubleshooting

You see	Fix
Toast: “Cannot reach the server”	Laravel isn’t running on port 8000 (`php artisan serve`).
Browser console: **CORS** error	Check `FRONTEND_URL` in Laravel’s `.env`, then run `php artisan config:clear`.
A new shipment stays “pending” forever	The queue worker isn’t running. Start `php artisan queue:work --queue=courier,notifications,default`.
Login says the credentials are wrong	Run `php artisan migrate --seed` in the Laravel project.
You’re logged out after a page refresh	The token was deleted or expired. Log in again.